TSM-AdvDaBa - Neo4j TP02

Jean-Daniel Küenzi - 04.05.2026

Table 1: Informations générales

ID du groupe	KuenziAdvDaBa2026
Lien vers le repository Git	https://github.com/jdkuenzi/tsm-advDaBa-2026
Namespace Kubernetes	kuenz-adv-daba-26
ID du pod Neo4j	kuenz-adv-daba-26-neo4j-0
Credentials Neo4j	neo4j/test
ID du pod de chargement	kuenz-adv-daba-26-insert-job-c4fjl

Si le pod de chargement n'est plus accessible sur le cluster, les logs sont également disponible à la racine du repo git.

1 Architecture

Ma solution est architecturée autour d'un script Python (main.py) qui se charge de lire le `JSONL` fourni en streaming (depuis une URL) et de l'insérer dans la base de données Neo4j. Ce script est encapsulé dans une image Docker (jdkuenzi/neo4jtp:latest) qui est déployée sur Kubernetes (k8s) sous la forme d'un Job à exécution unique.

La base de données Neo4j est déployée via un StatefulSet Kubernetes associé à un PersistentVolumeClaim de 25Gi pour garantir la persistance des données.

Les variables d'environnement (comme `JSON_URL`, `MAX_NODES`, `BATCH_SIZE`, etc.) sont gérées dynamiquement à l'aide d'un ConfigMap, ce qui permet d'ajuster le comportement du script d'insertion sans avoir à recompiler l'image Docker.

2 Insertion et optimisation

Afin de pouvoir insérer les millions de nœuds requis dans un temps acceptable et sans dépasser les limites matérielles strictes imposées (2 cœurs CPU et 3 Go de RAM), plusieurs choix techniques et optimisations ont été mis en place.

Il est important de préciser que j'ai pensé mon script pour être lancé une seule fois sur une base vide afin de pouvoir utiliser des `CREATE` sur les relations plutôt que des `MERGE`. Si l'on souhaite le relancer, il faut impérativement nettoyer la base avant.

2.1 Configuration de la mémoire Neo4j

Pour opérer efficacement dans les limites de ressources imposées (3 Go de RAM maximum), la configuration par défaut de Neo4j a été optimisée en ajustant l'unique variable suivante :

`dbms.memory.pagecache.size=1536m`.

Le page cache est la zone de mémoire que Neo4j utilise pour mettre en cache les données du graphe et les index, évitant ainsi de coûteux accès au disque. En lui allouant explicitement 1.5 Go (soit la moitié de la RAM totale disponible), on maximise la quantité de données conservées en mémoire rapide. Cela améliore considérablement les performances d'insertion : lors de l'exécution des nombreuses clauses `MERGE`, Neo4j doit constamment vérifier l'existence des nœuds via les index sur les `_id`. Grâce à ce cache élargi, ces recherches se font directement en RAM de manière quasi instantanée. De plus, ce réglage laisse suffisamment de marge (1.5 Go restants) pour le fonctionnement de la JVM et du système d'exploitation, prévenant ainsi les crashs liés à une saturation de la mémoire.

2.2 Gestion des coupures réseau

Le script est conçu pour tolérer les interruptions réseau et peut reprendre le traitement là où il s'est arrêté. Pour cela, il garde en mémoire et traque le dernier byte du flux. Lorsqu'une coupure survient, on attend un petit moment, puis on se reconnecte au serveur et on recommence le stream à partir de ce byte en passant dans l'en-tête `HTTP` la propriété "Range: bytes=xxx-" où xxx représente le byte où le flux doit reprendre. Un paramètre `MAX_RETRIES` contrôle le nombre de tentatives échouées successives de reconnexion au stream avant de l'interrompre définitivement.

2.3 Index

Avant toute insertion, le script commence par créer des contraintes d'unicité (`CREATE CONSTRAINT`) sur l'attribut `_id` des nœuds `ARTICLE` et `AUTHOR`. Cela a pour effet secondaire majeur de créer un index sur ces propriétés. Lors de l'utilisation des clauses `MERGE` et `MATCH`, l'indexation permet de réduire drastiquement la complexité de recherche (proche du O(1)) et empêche le temps d'insertion d'exploser.

2.4 Insertion en batch

Plutôt que d'insérer les nœuds un par un, les enregistrements `JSON` sont accumulés dans une liste (batch). Une fois que le lot atteint la taille de `BATCH_SIZE` (configurée via le ConfigMap), les données sont envoyées à Neo4j en une seule transaction.

2.5 Requêtes Cypher

Voici un exemple d'un article dans le `JSON`, je ne vais afficher que les champs qui nous intéressent :

```
{
  "id": "5390877920f70186a0d2ce7f",
  "title": "Top-Down Construction of 3-D Mechanical Object Shapes from
    Engineering Drawings",
  "authors": [
    {
      "id": "62aad3b2d9f2040d085dc9f9",
      "name": "H YOSHIURA",
      "org": "Hitachi Research Laboratory, Hitachi Ltd."
    },
    {
      "id": "5608ec4a45cedb3396db2920",
      "name": "K FUJIMURA",
      "org": "Univ. of Tokyo, Tokyo, Japan"
    },
```

```
    {
      "id": "548a62d3dabfae8a11fb49e2",
      "name": "TL KUNII",
      "org": null
    }
  ],
  "references": [
    "5390962020f70186a0df3bf2",
    "5390879d20f70186a0d43d74",
    "5390a1e620f70186a0e59c05",
    "5390b1d220f70186a0ee1bd6",
    "53909f8220f70186a0e3cfc7",
    "5390962020f70186a0df3f87",
    "53e9b8a8b7602d970447e86b"
  ],
  ...
}
```


L'enjeu principal de l'insertion réside dans le fait que les données sont fortement interconnectées : un article cite d'autres articles qui n'ont peut-être pas encore été lus par le script de chargement, ou encore l'article peut faire référence à un auteur via son id mais ne pas avoir le nom. Pour gérer cela sans perdre d'informations ni créer de doublons, j'ai divisé l'insertion d'un batch en trois étapes distinctes exécutées au sein d'une même transaction pour garantir l'intégrité et la performance.

Dans la première version du script, l'insertion s'effectuait via une unique requête Cypher complexe qui déléguait directement à Neo4j la tâche de parcourir et d'aplatir les listes imbriquées (auteurs et références) à l'aide de sous-requêtes CALL. Cette méthode concentrait toute la charge de calcul sur la base de données, ce qui exigeait énormément de mémoire pour maintenir le contexte d'exécution sur de gros volumes.

Dans la nouvelle version, cette logique a été repensée : c'est désormais le script Python qui se charge d'extraire et d'aplatir ces données complexes en amont pour générer des listes de relations simples. Ces listes pré-traitées sont ensuite transmises à Neo4j par sous-lots (chunking) avec des requêtes Cypher beaucoup plus légères (simples MATCH et CREATE). Ce changement de paradigme soulage considérablement le moteur de base de données, réduisant la complexité transactionnelle et prévenant efficacement les saturations mémoire ou les expirations de délai (timeouts).

A. Insertion des Articles

```
UNWIND $batch AS article
MERGE (a:ARTICLE {_id: article.id})
SET a.title = article.title
```

Cette première étape assure que l'article principal du bloc `JSON` existe en base de données et que sa propriété titre est à jour (par la suite, on verra que je crée des nœuds `ARTICLE` uniquement avec l'_id et non le titre). L'utilisation de `UNWIND` permet de traiter toute la liste du batch en une seule opération côté serveur.

B. Insertion des relations CITES

```
UNWIND $chunk AS edge
MATCH (src:ARTICLE {_id: edge.src})
MERGE (tgt:ARTICLE {_id: edge.tgt})
CREATE (src)-[:CITES]->(tgt)
```

Côté Python, les identifiants des références sont extraits pour générer une liste plate reliant la source à la cible. Si l'article cité dans les références n'existe pas encore dans la base de données, on crée un nœud `ARTICLE` avec uniquement son identifiant `_id` grâce au `MERGE`. L'étape A viendra mettre à jour son titre lorsque son entrée complète sera rencontrée plus tard. On s'appuie sur un `MATCH` du nœud source (déjà créé à l'étape A) et on forme la relation avec un simple `CREATE`.

C. Insertion des Auteurs et relation AUTHORED

```
UNWIND $chunk AS edge
MATCH (a:ARTICLE {_id: edge.article_id})
MERGE (au:AUTHOR {_id: edge.author_id})
SET au.name = edge.author_name
CREATE (au)-[:AUTHORED]->(a)
```

De la même manière, la liste des auteurs est aplatie côté Python pour former une liste liant un article à son auteur, avec le nom de ce dernier. Un auteur pouvant écrire plusieurs articles, on utilise `MERGE` sur le nœud `AUTHOR` afin d'éviter les doublons s'il existe déjà. L'instruction `SET` sert à mettre à jour son nom (utile si celui-ci manquait lors de la création initiale). Enfin, l'auteur est raccordé à son article, ciblé via un `MATCH`, grâce à la création d'une relation `AUTHORED`.

3 Performances observées

Le temps de chargement pour N+K nœuds (Articles et Auteurs) est tracé grâce aux logs générés par le logger Python qui affiche en fin d'exécution un résumé avec différentes statistiques. Les logs sont directement disponibles dans la console du pod.

```
2026-04-26 17:50:54 - INFO - =============== INSERTION FINIS ===============
2026-04-26 17:50:55 - INFO - Nombre total de noeuds lu en streaming : 1000000
2026-04-26 17:50:55 - INFO - N (Articles) : 3108245
2026-04-26 17:50:55 - INFO - K (Auteurs)  : 1242547
2026-04-26 17:50:55 - INFO - Total (N+K)  : 4350792 noeuds
2026-04-26 17:50:55 - INFO - start_time : 2026-04-26 17:24:46
2026-04-26 17:50:55 - INFO - end_time   : 2026-04-26 17:50:54
2026-04-26 17:50:55 - INFO - Temps de chargement total : 1568.52 secondes
2026-04-26 17:50:55 - INFO - ==============================================
```

Afin de comparer les performances, des tests ont été réalisés sur une machine locale via Docker et sur le cluster Kubernetes (k8s) avec différentes valeurs de `MAX_NODES`.

Table 2: Comparaison du temps de traitement, en secondes, pour différentes valeurs de `MAX_NODES` sur Docker et k8s

MAX_NODES	**Docker [s]**	**k8s [s]**
1000	1.8	5.55
10000	5.85	18.80
100000	56.97	131.12
1000000	551.06	1568.52

On observe tout d'abord que les temps de chargement en local avec Docker sont bien meilleurs que sur l'environnement Kubernetes malgré les restrictions dans le Docker Compose. Cette différence s'explique probablement par la surcharge réseau liée à l'architecture distribuée du cluster (communication entre le Pod Job et le Pod StatefulSet, coupures réseau) et par des performances d'écriture sur les volumes persistants (PVC k8s) potentiellement plus limitées que sur le disque

local de la machine de développement (SSD).

Néanmoins, la caractéristique la plus importante de cette logique d'insertion est sa croissance quasi linéaire. Que ce soit sur Docker ou k8s, multiplier le nombre de nœuds à traiter par 10 a pour conséquence de multiplier le temps d'exécution par environ 10. L'indexation initiale et l'usage de `UNWIND` prouvent ici leur efficacité, car le temps de recherche par `MERGE` reste constant malgré la taille grandissante du graphe.

En partant de cette évolution strictement linéaire observée sur 1'000'000 de nœuds, nous pouvons raisonnablement supposer que pour insérer les 10'000'000 de nœuds du dataset complet, le temps requis théorique sera de 10×1568.52 [s], soit 15685.20 secondes ($\sim$4h20min) sur Kubernetes. Bien sûr, cela ne tient pas compte des éventuelles chutes de performances liées aux coupures réseau ou encore à la surcharge des serveurs.

4 Exemple des données chargées

Après avoir streamé et inséré 1'500'000 de lignes du `JSONL` dans la base de données, j'ai exécuté la requête suivante pour valider l'insertion des nœuds dans le graphe.

```
MATCH (a1:AUTHOR {_id: "53f433c9dabfaedf43565a40"})-[:AUTHORED]->(art1:ARTICLE)
MATCH (a2:AUTHOR {_id: "53f43b47dabfaee4dc7b21d1"})-[:AUTHORED]->(art2:ARTICLE)

MATCH p = shortestPath((art1)-[:CITES]-(art2))

RETURN a1, p, a2
ORDER BY length(p) ASC
LIMIT 1
```

L'objectif de cette recherche est d'explorer le réseau pour trouver le chemin le plus court reliant un article écrit par le premier auteur (Marcelo Pasin) à un article écrit par le second auteur (Sébastien Rumley), en se basant exclusivement sur la chaîne des références bibliographiques qui séparent leurs travaux respectifs.

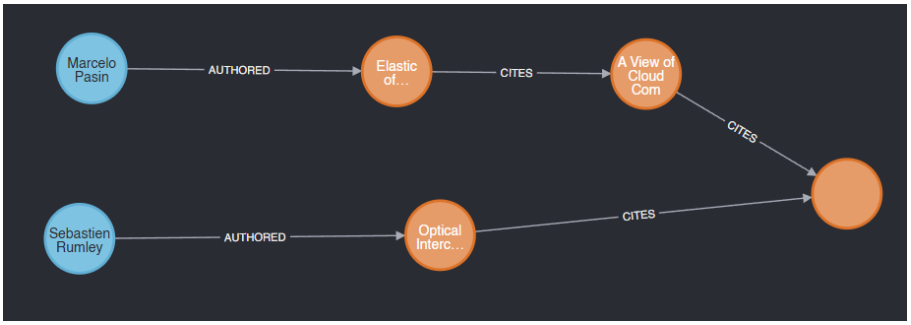

Figure 1: Plus court chemin entre Marcelo Pasin et Sébastien Rumley, basé sur la chaîne de références bibliographiques

Comme on peut le voir sur la figure 1, on retrouve bien un chemin entre les deux auteurs et on peut noter qu'un des articles cités n'a pas encore de titre. Comme la limite a été fixée à 1'500'000 de nœuds, cet article n'a pas encore été streamé, mais la requête a bien pu l'insérer dans le graphe avec ses relations.

5 Limites rencontrées à grande échelle

Malgré toutes les optimisations mises en place (indexation, batching, configuration Neo4j), une dégradation des performances a été observée lors des essais sur le cluster k8s avec plus de 1'500'000 de `MAX_NODES`. Lorsque le volume de documents insérés approche le seuil des 1'700'000 nœuds streamés, les opérations d'insertion commencent à prendre significativement plus de temps. Il est à noter qu'à partir de 1'500'000 de nœuds streamés, on compte dans la base 3'766'860 Articles et 1'588'485 Auteurs, soit 5'355'345 nœuds au total, sans compter les relations.

Au-delà de cette limite, des coupures de réseau commence à apparaître de manière fréquente. Je suppose que ce ralentissement interne à la base de données a pour conséquence de saturer le processus d'attente. Le temps de traitement des batchs s'allongeant, la connexion de streaming finit par expirer (timeout), entraînant des coupures réseau récurrentes.

Bien que le script soit robuste face à ces coupures et reprenne là où il s'est arrêté correctement, ces coupures réseau rallonge considérablement le temps d'insertions des articles. Cette perte de vitesse s'explique très probablement par la taille grandissante du graphe. En s'approchant de ce volume, les index et les données dépassent la capacité du page cache disponible (limité par les 3 Go de RAM et la configuration), forçant Neo4j à multiplier les accès disques beaucoup plus lents (`SWAP`).

6 Améliorations possibles

Étant donné que la configuration alloue 2 cœurs CPU, une piste d'amélioration consisterait à implémenter du multithreading. Un thread pourrait être spécifiquement dédié au téléchargement du flux de données et à la création des batchs, tandis que le second se chargerait de l'insertion en base. Il faudrait toutefois gérer la création des batchs : s'ils sont plus rapides que l'insertion, il est nécessaire de contrôler une file d'attente pour ne pas saturer la RAM qui reste une ressource très limitée.